# Highlights

## CoDA-FTP: Conservative Domain Adaptation for Cross-Project Flaky-Test Prediction

Yu Zhang, Yan Lei, Huan Xie

- We frame flaky-test prediction as cross-project deployment.
- We propose CoDA-FTP for new-project flaky-test prediction.
- We conduct extensive experiments under cross-project evaluation.
- CoDA-FTP achieves 70% precision, 54% F1, and 92% AUC.
- Ablations show adaptation cuts false positives from 2,141 to 138.

Preprint not peer reviewed

This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6963911

# CoDA-FTP: Conservative Domain Adaptation for Cross-Project Flaky-Test Prediction

Yu Zhang[a], Yan Lei[a,*] and Huan Xie[a]

[a]School of Big Data and Software Engineering, Chongqing University, Chongqing, 401331, China

ARTICLE INFO

*Keywords*:
flaky tests
cross-project prediction
domain adaptation
software testing
source selection

ABSTRACT

Flaky tests undermine continuous integration feedback because their outcomes can change without code changes. Flaky-test prediction is especially useful before a project has accumulated enough local flakiness history to train a reliable project-specific model. This paper studies cross-project flaky-test prediction as a deployment-oriented setting, where knowledge learned from existing projects is applied to a target project whose tests follow a different distribution. We propose CoDA-FTP, a conservative domain-adaptation approach for this setting. CoDA-FTP builds fused expert and semantic test representations, selects source projects that are close to the target project, and adapts only the selected source representations toward the target feature distribution before prediction. This select-then-align design aims to reduce negative transfer from distant projects while avoiding over-expansion of flaky warnings. On 21,413 tests from 23 Java projects under leave-one-project-out evaluation, CoDA-FTP achieves 70% precision, 44% recall, 54% F1, and 92% AUC. Compared with traditional feature-based and neural baselines, the improvement mainly comes from reducing false positives while preserving useful recall. These results show that conservative selected-source adaptation is a practical design principle for cross-project flaky-test prediction.

## 1. Introduction

A flaky test is a test whose outcome can change across executions even when the code under test and the test code remain unchanged [1, 2]. This nondeterminism weakens continuous integration (CI), where test outcomes are expected to serve as development signals and a failing test should indicate that the current change may have broken the system. Prior empirical studies [3, 4, 5, 6] show that flaky tests are common in open-source and industrial projects, arise from diverse causes such as asynchronous behavior, test-order dependence, UI nondeterminism, and resource interference, and impose substantial costs on debugging and regression testing workflows. Existing techniques therefore aim either to detect flaky tests through additional executions, partial classification, coverage-based reasoning, or rerun-and-learning hybrids [7, 3, 8], or to predict flakiness before repeatedly rerunning every test [9, 10, 11, 12].

The practical value of flaky-test prediction depends strongly on when it can be deployed. Existing predictors show that flakiness signals can be learned from expert testing features, vocabulary information, and learned code representations [9, 10, 11]. However, a project-specific predictor assumes that the project to be predicted has already accumulated enough flakiness history to learn from. This assumption is limiting in early deployment scenarios, such as when a project is newly added to CI, a test suite is newly introduced or substantially changed, or developers want to screen tests before enough project-specific evidence has accumulated. In such cases, the model faces tests from a project it has not previously seen.

*Corresponding author
yanlei@cqu.edu.cn (Y. Lei)
ORCID(s): 0009-0007-2857-8890 (Y. Zhang); 0000-0003-4504-6806 (Y. Lei)

This early-deployment concern is not unique to flaky-test prediction. Cross-project defect prediction was introduced for projects with limited local defect data, where *source projects* are historical projects with labeled examples used for training and the *target project* is the project on which the trained model is applied [13, 14, 15]. More recent cross-project defect prediction studies continue to examine source selection, multi-source transfer, and domain adaptation because external projects are not uniformly useful for a target project [16, 17]. Online just-in-time defect prediction reaches a similar conclusion from a deployment perspective because training data arrives over time and cross-project data can be especially useful during the initial phase of a project [18, 19]. These studies do not solve flaky-test prediction, but they establish a common software-engineering lesson that evaluation should reflect the amount of project-specific history available at deployment time.

For flaky-test prediction, this lesson implies that cross-project prediction is not merely a stricter benchmark variant. It is the setting in which a predictor must help before the target project has enough local flakiness history. Evidence from mixed-project evaluation therefore does not by itself establish deployability on an unseen project. When training and test instances can come from the same project pool, a model may benefit from project-specific regularities that are unavailable when an entire target project is held out. Recent work [20] on flaky test classification makes the same point by arguing for project-wise disjoint evaluation, since tests from the same project may share structural or contextual similarities and project-based splits better simulate applying a classifier to a new project. Cross-project flaky-test prediction therefore asks whether useful flaky-test signals can transfer across project boundaries while keeping warnings calibrated for the target project.



This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6963911

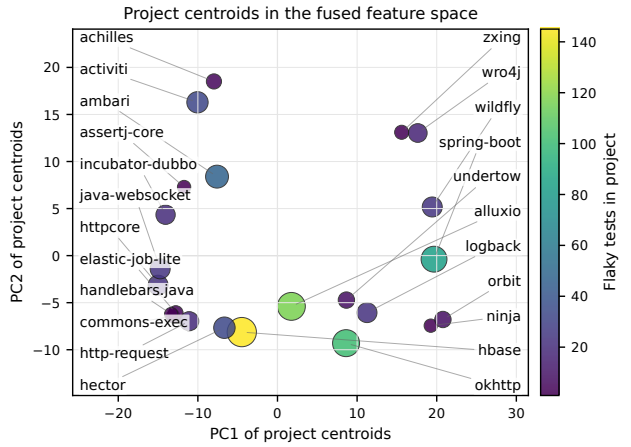



**Figure 1:** Project-level distribution shift in the fused feature space. Each point represents one project centroid after standardizing the 790-dimensional fused test representation and projecting project centroids onto the first two principal components. Point color and size indicate the number of flaky tests in the project.

However, cross-project prediction changes the main technical difficulty. The challenge is not simply to train a stronger classifier, but to transfer knowledge across project-level domains. Test suites differ in code semantics, testing idioms, APIs, naming conventions, dependency structure, build systems, and execution environments. As a result, feature distributions observed in source projects may not match those in a target project, a distribution-mismatch problem that prior cross-project defect prediction research has repeatedly shown to degrade transfer [13, 14, 15, 21, 22, 16, 17]. We observe the same project-level mismatch in the flaky-test data used in our study. To make the shift concrete, we summarize each project by a *project centroid*, i.e., the average vector of all tests in that project after feature standardization. Figure 1 shows that project centroids occupy visibly different regions of the fused feature space, and Table 1 shows that, for a typical target project, the farthest candidate source is 4.36× farther than the closest candidate source. This evidence supports treating each project as a deployment domain rather than as an exchangeable subset of one homogeneous dataset. In flaky-test prediction, this mismatch is particularly damaging because flaky tests are rare, and a transferred model can over-generalize source-specific flaky patterns and produce excessive false positives on ordinary target-project tests. We refer to this failure mode as *false-positive amplification*.

This evidence motivates a conservative transfer strategy. A cross-project predictor must decide which source projects are reliable for the target and how to adapt those sources without expanding warnings for the rare flaky class. Directly pooling all sources can mix transferable examples with project-specific artifacts, while adapting an indiscriminate source pool can make transfer more confident without making it more reliable.

**Table 1**
Project-level distribution shift in the fused feature space. Distances are computed between standardized project centroids; lower values indicate more similar projects.

| Measurement | Value |
|---|---|
| Number of projects | 23 |
| Typical distance between two projects | 30.73 |
| Largest distance between two projects | 44.07 |
| Typical closest source to a target | 9.45 |
| Typical farthest source to a target | 39.66 |
| Typical farthest/closest gap | 4.36× |
| Largest farthest/closest gap | 22.84× |

We propose *CoDA-FTP*, a conservative domain-adaptation approach for cross-project flaky-test prediction. CoDA-FTP instantiates conservative selected-source adaptation through a select-then-align design that first chooses the top-$k$ historical source projects closest to the target project in the fused feature space and then aligns the feature distribution of only those selected projects to the target. It represents each test using a fused feature vector that combines expert testing features with CodeBERT [23] semantic representations, performs target-aware source selection in this fused space, applies CORAL [24] to match selected-source second-order statistics to the target, and trains a regularized classifier with mild source-side imbalance handling. This design makes transfer selective at the project level and conservative at the prediction level because distant sources are excluded before alignment, and adapted source evidence is converted into target warnings through a high-precision classifier.

We evaluate CoDA-FTP under a leave-one-project-out protocol, where each project is treated as an unseen target in turn. This protocol approximates the deployment scenario in which a predictor trained from existing projects is applied to a project that has not contributed training history. Our evaluation compares CoDA-FTP with traditional feature-based predictors and neural code-model predictors, and includes a bounded reference comparison with zero-shot and source-only few-shot LLM prompting. The results show that CoDA-FTP improves cross-project flaky-test prediction primarily by reducing false positives while preserving useful recall. Component ablations further show that target-aware source selection and selected-source adaptation address complementary failure modes: source selection reduces negative transfer from distant projects, whereas adaptation controls false-positive amplification under residual source-target mismatch.

This paper makes the following contributions.

- We establish cross-project flaky-test prediction as a deployment-oriented evaluation setting and study it on



This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6963911

23 Java projects using leave-one-project-out prediction, where each target project is held out as unseen training history.

- We propose CoDA-FTP for cross-project flaky-test prediction. CoDA-FTP operationalizes conservative selected-source adaptation by selecting transferable historical projects and adapting their test representations, reducing the risk that source-specific flaky patterns become false warnings in the target project.
- We provide a broad empirical study against traditional, neural/code-model, transfer-learning, and prompt-only LLM predictors. CoDA-FTP achieves 70% precision, 44% recall, 54% F1, and 92% AUC, while component analysis shows that selected-source adaptation reduces false positives from 2,141 to 138.

The remainder of this paper is organized as follows. Section 2 reviews related work. Section 3 presents CoDA-FTP. Section 4 describes the experimental design, and Section 5 reports the results. Section 6 discusses threats to validity, and Section 7 concludes the paper.

## 2. Related Work

### 2.1. Domain Adaptation and Source Selection

Domain adaptation has been widely studied as a way to address distribution mismatch between source and target domains [25, 26]. Its objective is to reduce the gap between training and deployment feature distributions, so that a model trained on one domain can generalize to another. This objective is especially relevant to cross-project software engineering prediction, where projects may differ in coding style, APIs, process, execution environment, and testing practice. Prior work in cross-project defect prediction has therefore explored transfer learning, training-data selection, and benchmark designs that account for project-level mismatch [13, 14, 15, 22].

Existing domain adaptation methods follow several broad strategies. Some learn domain-invariant representations through adversarial objectives or representation-learning objectives, such as domain-adversarial neural networks, adversarial cross-project defect prediction models, transfer component analysis, and deep adaptation networks [27, 17, 28, 29]. Others reduce statistical discrepancies, for example through maximum mean discrepancy or second-order covariance matching [30, 24]. CORAL belongs to the latter family and matches second-order feature statistics between source and target data [24]. This lightweight unsupervised mechanism fits our setting because target-project test features are observable at deployment time, while project-specific flaky labels are not available for supervised training.

However, direct alignment alone is not sufficient for cross-project flaky-test prediction under conservative deployment goals. If the source pool contains projects that are far from the target, aligning all sources may still propagate project-specific artifacts and amplify false positives. Training-data selection addresses a complementary issue by choosing source instances or projects that are more relevant to the target before model training [13, 31, 32, 22]. CoDA-FTP combines these ideas in a select-then-align design. It first performs target-aware source selection to reduce coarse project-level mismatch, and then applies CORAL only to the selected sources to reduce residual feature-level shift. The gap is that reducing discrepancy is not enough under a rare flaky class because transfer is harmful if it enlarges false-positive warnings. Thus, CoDA-FTP connects flaky-test prediction, cross-project deployment, and domain adaptation through a conservative selected-source adaptation strategy rather than through direct transfer or blind alignment.

### 2.2. Cross-Project Prediction in Software Engineering

Cross-project prediction is a long-standing problem in software engineering. Its central motivation is practical because external projects may provide the only available training experience when a target project lacks enough local history. This setting has been extensively studied in defect prediction, where researchers have investigated whether models trained on one project, company, or domain can predict defects in another [33, 13, 14, 22]. Later work has also explored compositional and multi-source transfer designs for improving cross-project defect prediction under heterogeneous source pools [34, 16]. Systematic reviews show that conclusions are sensitive to datasets, metrics, model families, and preprocessing choices [35, 36]. These studies show that cross-project prediction is useful but difficult because project differences can substantially affect transfer performance.

To improve cross-project transfer, prior work has explored two main directions. One selects or filters training data before model construction, based on the intuition that only part of the external data is relevant to a target project [13, 31, 37, 32]. Such methods reduce negative transfer by excluding distant source instances, but they can discard useful signal when relevance is estimated too narrowly. Another direction adapts the representation, weights, or distribution of source data, such as transfer Naive Bayes, transfer defect learning, heterogeneous defect prediction, CCA-based transfer for heterogeneous metrics, and multi-source transfer frameworks [14, 15, 38, 21, 16]. These methods show that mismatch can be addressed by transforming or reweighting source information, but adapting irrelevant source data can still propagate source-specific artifacts.

CoDA-FTP borrows this high-level lesson but applies it to a different prediction object. Prior cross-project defect prediction work usually predicts defective modules or changes, whereas CoDA-FTP predicts flaky tests. This distinction matters because the positive class in flaky-test prediction is rare and false positives create a direct triage burden, since an over-transferred model can ask developers to inspect many ordinary tests as potentially flaky. Similar concerns also arise in online just-in-time defect prediction, where cross-project learning is especially relevant during



This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6963911

the early phase of a project [18, 19]. CoDA-FTP therefore treats cross-project flaky-test prediction as a deployment-time transfer problem and combines the two cross-project transfer routes by first selecting target-relevant source projects and then adapting only those selected sources to the target distribution. Unlike classic cross-project defect prediction settings, the deployment cost here is shaped not only by missed positives but also by warning burden under extreme imbalance.

### 2.3. Flaky Test Detection and Prediction

Research on flaky tests has studied their prevalence, causes, taxonomy, and lifecycle in both open-source and industrial projects [1, 3, 4, 6, 2]. Other empirical studies show that flakiness is not limited to traditional unit tests; UI tests also exhibit diverse manifestation and fixing patterns [5]. Based on these observations, prior work has developed techniques for detecting flaky tests through reruns, coverage changes, execution evidence, or partial classification [7, 3, 8]. Recent work also shows that execution-failure semantics can affect how flaky-test prediction should be interpreted in practice [12]. A complementary line of work predicts flakiness without repeatedly executing every test, using expert testing features, vocabulary or language-model signals, hybrid representations, ranking objectives, and more fine-grained classification formulations [9, 10, 11, 39, 40]. These studies show that flaky tests expose learnable signals and that different prediction objectives can support test prioritization or diagnosis. CoDA-FTP builds on this line of work but focuses on the question of how such signals can be transferred conservatively when the predictor is deployed on a project that differs from the projects used for training.

## 3. Method

CoDA-FTP is designed for cross-project flaky-test prediction, where a predictor must apply knowledge learned from existing projects to a target project that it has not previously seen. The method follows the conservative selected-source adaptation principle introduced earlier: before aligning source data to the target, it first restricts transfer to source projects that are likely to be relevant. Figure 2 summarizes this select-then-align pipeline.

### 3.1. Problem Setting

Let $S = \{S_1, \ldots, S_m\}$ denote the set of candidate source projects and let $T$ denote a target project. Each project contains test methods represented in a common feature space. In a cross-project setting, CoDA-FTP uses the source projects as training experience and predicts whether tests in $T$ are flaky. The leave-one-project-out protocol used in our evaluation instantiates this setting by holding out one project as the target and treating all remaining projects as candidate sources.

The goal is to learn a target-project predictor $f_T$ that assigns each target test a flaky probability. Unlike within-project prediction, this setting evaluates whether flakiness patterns learned from existing projects can transfer to a project whose testing distribution may differ from the sources. The main failure mode is false-positive amplification: source-specific patterns that are predictive in one project may correspond to ordinary tests in another project. CoDA-FTP addresses this risk by making transfer selective before making it adaptive.

### 3.2. Overview of CoDA-FTP

To address false-positive amplification under cross-project distribution shift, CoDA-FTP organizes flaky-test prediction as a conservative selected-source adaptation pipeline. As illustrated in Figure 2, the framework consists of three stages. In the *Data and Features* stage, CoDA-FTP extracts heterogeneous test information and concatenates it into a fused representation. This representation combines FlakeFlagger expert testing features with semantic code representations, producing a common space in which source and target tests can be compared. In the *Domain Adaptation* stage, CoDA-FTP first selects source projects that are close to the target project and then aligns only those selected sources to the target distribution with CORAL. In the *Prediction and Inference* stage, CoDA-FTP trains a regularized classifier on the aligned selected sources and predicts flaky and non-flaky tests in the target project.

The three stages are designed to address different sources of cross-project risk. The fused representation provides the substrate for both similarity estimation and classifier training. Source selection controls coarse project-level mismatch by deciding which source projects should enter transfer. CORAL alignment controls residual feature-level mismatch by re-expressing the selected source features in a geometry closer to the target distribution. Conservative classifier training then avoids over-expanding the rare flaky class after adaptation. Together, these components make transfer selective before making it adaptive, and make adaptation conservative before producing target-project predictions.

We use *conservative* to mean that the pipeline is designed to avoid expanding the predicted flaky region unless the source evidence is both target-relevant and distributionally adapted. Operationally, CoDA-FTP implements this principle by restricting the source pool to target-relevant projects, aligning only the selected sources, using mild source-side imbalance handling, and applying a high-precision operating threshold. Conservativeness is therefore not a separate module; it is the design constraint that connects source selection, alignment, and final inference.

### 3.3. Data and Features

The first stage prepares a unified feature space for cross-project comparison and adaptation. For each test $j$, CoDA-FTP constructs a fused representation

$$\mathbf{x}_j = [\mathbf{e}_j; \mathbf{z}_j],$$

where $\mathbf{e}_j$ denotes expert testing features and $\mathbf{z}_j$ denotes a semantic representation of the test code. The expert part $\mathbf{e}_j$ uses the FlakeFlagger features released with the public



This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6963911

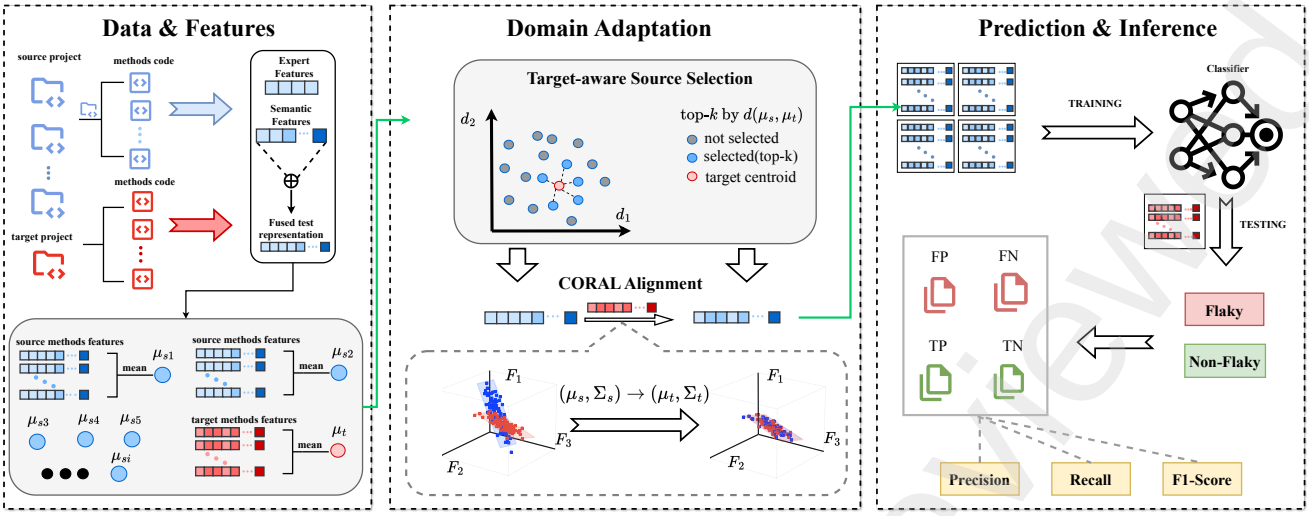



**Figure 2:** Overview of CoDA-FTP. CoDA-FTP constructs fused test representations, selects target-relevant source projects, aligns the selected sources to the target distribution with CORAL, and trains a conservative classifier for cross-project flaky-test prediction.

FlakeFlagger dataset [9]. These features capture testing-related properties such as test smells and structural characteristics that were designed for flakiness prediction. Reusing these expert features keeps CoDA-FTP comparable with the original dataset setting and with later flaky-test predictors, including DeepFlaky [11]. Similar feature-fusion designs have also been used in testing and debugging tasks where expert-style signals and semantic representations capture complementary evidence [41].

The semantic part $\mathbf{z}_j$ is a CodeBERT representation of the test method [23]. CoDA-FTP encodes the final test-method code using CodeBERT without task-specific fine-tuning. Inputs are padded or truncated to 510 tokens, and $\mathbf{z}_j$ is the last-layer representation of the first token, i.e., the CodeBERT `[CLS]` position. In the current implementation, $\mathbf{e}_j$ contains 22 expert features and $\mathbf{z}_j$ contains 768 semantic dimensions, yielding a 790-dimensional fused vector. The fused vector serves two purposes. First, it provides a shared coordinate system for estimating how close each candidate source project is to the target. Second, it serves as the feature matrix on which CORAL alignment and classifier training operate. This avoids using one representation for source selection and a different representation for prediction, which could make the adaptation decision inconsistent with the downstream classifier.

## 3.4. Domain Adaptation

### *3.4.1. Target-Aware Source Selection*

The second stage begins by deciding which historical projects should be transferred. Not all source projects are equally useful for a target project, a finding that also motivates training-data selection in cross-project defect prediction [13, 31, 32]. Directly pooling all available sources can introduce project-specific artifacts from distant projects and increase negative transfer. CoDA-FTP therefore first selects source projects whose fused feature distributions are closest to the target project. The input to this step is the candidate source pool $S$ and the target feature matrix; the output is a selected subset of source projects for subsequent alignment.

For each target project, CoDA-FTP excludes the label column and metadata columns from the selection computation, including the flaky-test label, test name, and project identifier. Let $\mathbf{X}_{all}$ be the feature matrix obtained by merging all candidate source projects. CoDA-FTP computes source-side feature statistics

$$\boldsymbol{\mu}_{all} = \mathrm{mean}(\mathbf{X}_{all}), \qquad \boldsymbol{\sigma}_{all} = \mathrm{std}(\mathbf{X}_{all}) + \epsilon,$$

where $\epsilon$ avoids division by zero. It then standardizes the target project and each candidate source project using these source-side statistics and computes their centroids:

$$\mathbf{c}_t = \mathrm{mean}\left(\frac{\mathbf{X}_t - \boldsymbol{\mu}_{all}}{\boldsymbol{\sigma}_{all}}\right), \qquad \mathbf{c}_i = \mathrm{mean}\left(\frac{\mathbf{X}_i - \boldsymbol{\mu}_{all}}{\boldsymbol{\sigma}_{all}}\right).$$

The distance between source project $S_i$ and target project $T$ is

$$d(S_i, T) = \|\mathbf{c}_i - \mathbf{c}_t\|_2.$$

CoDA-FTP ranks candidate sources by this distance and selects the top-$k$ closest projects, with a minimum number of selected projects to avoid an overly small training pool. The main configuration uses a fixed closest-quarter rule: with 22 candidate source projects in each leave-one-project-out fold, it selects the six closest projects and enforces a minimum of three projects. This step reduces coarse project-level mismatch before any distribution alignment is applied. In effect, source selection constrains the adaptation problem: CORAL is not asked to align every historical project to the target, but only those sources that already provide a plausible transfer basis.



This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6963911

#### 3.4.2. Selected-Source CORAL Alignment

After source selection, the remaining problem is how to adapt the selected source data to the target project. Source selection reduces negative transfer from distant projects, but the selected sources may still differ from the target at the feature-distribution level. CoDA-FTP therefore applies CORAL to align the selected source feature matrix to the target feature distribution. CORAL is an unsupervised domain alignment method that matches second-order statistics across domains [24]. In our pipeline, CORAL takes the selected source feature matrix and the target feature matrix as input, and outputs an aligned source matrix used for classifier training.

Let $\mathbf{X}_s \in \mathbb{R}^{n_s \times d}$ denote the merged feature matrix of the selected source projects, and let $\mathbf{X}_t \in \mathbb{R}^{n_t \times d}$ denote the target feature matrix. CORAL aligns domains by reducing the discrepancy between their covariance matrices. Its alignment loss can be written as

$$\mathcal{L}_{CORAL} = \frac{1}{4d^2} \left\| \mathbf{C}_s - \mathbf{C}_t \right\|_F^2,$$

where $d$ is the feature dimension, $\|\cdot\|_F$ is the Frobenius norm, and $\mathbf{C}_s$ and $\mathbf{C}_t$ are the source and target covariance matrices. In CoDA-FTP, this objective is applied after source selection, so the source covariance is computed only from the selected source projects rather than from all historical projects.

CoDA-FTP computes source and target means $\boldsymbol{\mu}_s$ and $\boldsymbol{\mu}_t$, centers the two feature matrices, and estimates regularized covariance matrices:

$$\widetilde{\mathbf{C}}_s = \mathrm{cov}(\mathbf{X}_s) + \lambda \mathbf{I}, \qquad \widetilde{\mathbf{C}}_t = \mathrm{cov}(\mathbf{X}_t) + \lambda \mathbf{I},$$

where $\lambda$ is a small diagonal regularizer. The CORAL transformation is then

$$\mathbf{A}_{CORAL} = \widetilde{\mathbf{C}}_s^{-1/2} \widetilde{\mathbf{C}}_t^{1/2}.$$

The aligned source matrix is

$$\mathbf{X}_s' = (\mathbf{X}_s - \boldsymbol{\mu}_s)\mathbf{A}_{CORAL} + \boldsymbol{\mu}_t.$$

The transformation first whitens the selected source distribution with $\widetilde{\mathbf{C}}_s^{-1/2}$ and then recolors it with the target covariance structure $\widetilde{\mathbf{C}}_t^{1/2}$. Intuitively, this step changes how selected source tests are expressed in the fused feature space, so that their covariance geometry better matches the target project before the classifier is trained. In implementation, CoDA-FTP uses $\lambda = 10^{-3}$ to improve numerical stability in the high-dimensional fused feature space. CoDA-FTP does not modify the CORAL algorithm itself; it integrates CORAL into a conservative selected-source adaptation pipeline where alignment is applied only after target-aware source selection.

### 3.5. Prediction and Inference

The final stage turns the adapted source representation into target-project predictions. After alignment, CoDA-FTP trains a regularized XGBoost classifier [42] on the aligned selected source features and source labels. Because flaky tests are rare, the classifier should learn transferable flaky-test signals without aggressively expanding the positive region. CoDA-FTP therefore uses mild source-side imbalance handling rather than treating oversampling as a central mechanism. In the main configuration, weak SMOTE [43] is applied after CORAL alignment and before classifier training.

The classifier outputs a flaky probability for each target test. CoDA-FTP then applies a fixed decision threshold to obtain binary flaky/non-flaky predictions. The threshold, classifier hyperparameters, and exact evaluation protocol are specified in the experimental design so that the method description focuses on the adaptation mechanism rather than on protocol tuning.

## 4. Experimental Design

This section describes how we evaluate CoDA-FTP. The evaluation is designed around the deployment scenario introduced earlier, in which a model is trained from existing projects and then applied to a held-out project whose tests have not been used for supervised training.

### 4.1. Research Questions

We study four research questions.

**RQ1: How effective is CoDA-FTP against traditional and neural flaky-test predictors?**

**RQ2: How does CoDA-FTP compare with prompt-only LLM predictors?**

**RQ3: Which design choices and transfer operators explain CoDA-FTP's cross-project behavior?**

**RQ4: How stable is CoDA-FTP under deployment choices?**

These RQs separate comparison, mechanism, and robustness. RQ1 evaluates overall effectiveness against established non-LLM flaky-test predictors. RQ2 provides a bounded reference comparison with prompt-only code LLMs, rather than treating direct prompting as a fully trained competing pipeline. RQ3 examines which design choices and transfer operators explain CoDA-FTP's behavior, and RQ4 checks sensitivity to key deployment choices.

### 4.2. Dataset and Features

We evaluate CoDA-FTP on the public FlakeFlagger Java flaky-test dataset collected by Alshammari et al. [9]. This dataset provides project-level test instances and expert flakiness features, and it has also been used by later flaky-test prediction studies such as DeepFlaky [11]. We use the same project-level dataset, but reorganize the evaluation as leave-one-project-out prediction to test cross-project deployment.

Table 2 summarizes the data used in our evaluation. The dataset contains 21,413 tests from 23 projects, including 724 flaky tests and 20,689 non-flaky tests. This distribution is highly imbalanced, with flaky tests accounting for only 3.38% of the instances. Such imbalance is expected in flaky-test prediction, but it also makes false positives a practical concern because even a small false-positive rate can produce



This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6963911

**Table 2**
Dataset composition by project.

| Project | Tests | Flaky | Non-flaky | Project | Tests | Flaky | Non-flaky |
|---|---|---|---|---|---|---|---|
| achilles | 1,316 | 4 | 1,312 | incubator-dubbo | 2,133 | 19 | 2,114 |
| activiti | 2,041 | 32 | 2,009 | java-websocket | 145 | 23 | 122 |
| alluxio | 187 | 116 | 71 | logback | 803 | 22 | 781 |
| ambari | 313 | 46 | 267 | ninja | 307 | 1 | 306 |
| assertj-core | 6,205 | 1 | 6,204 | okhttp | 808 | 100 | 708 |
| commons-exec | 55 | 1 | 54 | orbit | 86 | 7 | 79 |
| elastic-job-lite | 558 | 3 | 555 | spring-boot | 2,017 | 82 | 1,935 |
| handlebars.java | 351 | 1 | 350 | undertow | 181 | 7 | 174 |
| hbase | 429 | 145 | 284 | wildfly | 993 | 23 | 970 |
| hector | 131 | 33 | 98 | wro4j | 1,134 | 16 | 1,118 |
| http-request | 163 | 18 | 145 | zxing | 345 | 2 | 343 |
| httpcore | 712 | 22 | 690 | | | | |
| **Total** | **21,413** | **724** | **20,689** | | | | |

many incorrect flaky warnings. This is why imbalanced classification studies often emphasize precision–recall behavior in addition to ROC-based ranking metrics [44, 45].

Each test is represented using the fused representation described in Section 3. The representation contains 22 FlakeFlagger expert testing features and a 768-dimensional CodeBERT semantic vector [23], resulting in 790 features per test. Project identifiers and test names are used only for grouping, splitting, and reporting; they are excluded from source selection, alignment, and classification.

### 4.3. Baselines

We compare CoDA-FTP with representative flaky-test prediction baselines and prompt-only LLM reference methods.

**Traditional feature-based baselines. FlakeFlagger** uses expert features for static flaky-test prediction [9]. **Vocabulary + FlakeFlagger** augments those expert features with vocabulary information from test code, following prior work on vocabulary-based flaky-test prediction [46]. We also include a **FLAST-style** static-neighbor baseline, following FLAST's idea of predicting flakiness from test-code similarity [47].

**Neural/code-model baselines. Flakify** uses CodeBERT-based semantic information for black-box flaky-test prediction [10]. **DeepFlaky** combines semantic representations with contrastive learning for flaky-test prediction [11]; we include it as a neural baseline under the same leave-one-project-out target projects.

**Transfer-learning diagnostics.** Because CoDA-FTP is motivated by cross-project distribution shift, we include diagnostic transfer variants that represent different adaptation strategies and source scopes. Transfer Component Analysis (TCA) learns a latent representation to reduce source–target distribution discrepancy [28]. We evaluate an all-source TCA variant that adapts all candidate source projects and a selected-source TCA variant that keeps CoDA-FTP's target-aware source selection fixed and replaces only CORAL. We also include a selected-source weighting variant that estimates source weights using a source-vs-target domain classifier. These diagnostics test whether CoDA-FTP's behavior comes from conservative selected-source adaptation rather than from a single alignment algorithm.

**Prompt-only LLM reference methods.** We additionally evaluate prompt-only LLM predictors as a reference point for direct use of modern code LLMs. The completed LLM comparisons use Qwen2.5-Coder-1.5B and Qwen2.5-Coder-7B [48], and Qwen3.6-35B from the Qwen3 model family [49]. For each model family, we consider zero-shot prompting with a neutral instruction and source-only four-shot prompting. The four-shot setting uses two flaky and two non-flaky examples sampled only from source projects for the current target fold.

We use prompt-only LLM reference methods rather than training an LLM on all source projects for each held-out target. Including all source tests in the prompt is infeasible, and fold-specific fine-tuning would introduce a substantially different training pipeline, computational cost, and checkpoint-selection problem. RQ2 therefore evaluates a lightweight deployment mode in which a code LLM predicts target tests directly, optionally with a few source-project demonstrations; it is not an upper bound on fine-tuned, retrieval-augmented, or calibrated LLM pipelines.

### 4.4. Baseline Settings

All completed baselines are evaluated under the same leave-one-project-out split as CoDA-FTP. For FlakeFlagger and Vocabulary + FlakeFlagger, feature selection uses source projects only in each fold. FlakeFlagger uses the official expert feature families with a random forest classifier [50], SMOTE, information-gain threshold 0.01, and 5,000 trees. Vocabulary + FlakeFlagger augments those expert features with source-selected vocabulary features. The FLAST-style baseline uses test-code tokens, cosine $k$-nearest-neighbor scoring, and the main FLAST settings $k \in \{3, 7\}$ and $\sigma \in \{0.5, 0.95\}$. Flakify and DeepFlaky are evaluated on the same project-level instances and held-out target



This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6963911

projects. The transfer diagnostics use the same fused representation, XGBoost configuration, imbalance handling, and threshold as CoDA-FTP. TCA applies a 128-dimensional linear latent space, and source weighting uses clipped and normalized weights from a logistic source-vs-target domain classifier.

For the prompt-only LLM reference methods, zero-shot prompting gives the definitions of flaky and non-flaky tests, presents the target Java test, and asks the model to return exactly one label without explanation. The source-only four-shot prompt uses the same task definition and prepends four source-project examples, including two flaky and two non-flaky tests sampled from the source projects of the current fold. We keep these templates fixed across LLMs, changing only whether source examples are included; the complete prompts are included in the replication package. The Qwen3.6-35B experiments are run with thinking disabled. LLM outputs are parsed into binary flaky/non-flaky decisions, so these prompt-only methods do not provide calibrated scores for AUC.

### 4.5. Evaluation Protocol

We use leave-one-project-out evaluation. In each fold, one project is held out as the target project, and the remaining 22 projects are used as candidate source projects. CoDA-FTP first extracts fused representations for source and target tests, then selects target-relevant source projects, applies its selected-source adaptation step, trains on the adapted source instances, and predicts flaky tests in the held-out project.

The labels of the held-out project are reserved for final evaluation. They are not used within a fold for source selection, source adaptation, class balancing, classifier training, or probability estimation. The reported decision threshold is treated as part of the fixed evaluation protocol rather than as a fold-specific learned parameter. Because CoDA-FTP targets an actionable-warning setting, this fixed threshold is interpreted as a conservative operating point. Among near-equivalent F1 choices, the protocol favors fewer false positives and higher precision. RQ4 reports threshold sensitivity to show how the reported operating point relates to nearby alternatives. We aggregate predictions from all 23 held-out folds and compute corpus-level metrics from the aggregated confusion matrix. This aggregation reflects the overall deployment behavior of a cross-project predictor across projects of different sizes.

### 4.6. Metrics

We report precision, recall, F1, AUC, and the confusion-matrix counts TP, FN, FP, and TN. Precision measures how many predicted flaky tests are truly flaky, while recall measures how many flaky tests are detected. F1 summarizes the precision–recall trade-off, and AUC measures threshold-independent ranking ability from the predicted probabilities [51, 44]. We compute AUC from the aggregated out-of-fold prediction scores under the same leave-one-project-out protocol used for the confusion matrix when per-test scores are available. For probabilistic methods with per-test scores, we also report PR-AUC and recall under fixed false-positive budgets. These diagnostics test whether the ranking remains useful when deployment constrains the number of warnings that developers can inspect. For methods that do not provide calibrated per-test scores in our completed runs, including prompt-only LLM methods, we omit AUC, PR-AUC, and fixed-budget recall.

Precision and false positives are central in this setting. A cross-project predictor is useful only if its warnings are credible enough for developers to inspect, rerun, or prioritize tests. Because non-flaky tests are much more common than flaky tests, over-predicting the rare flaky class can quickly create a large number of incorrect warnings. This concern is consistent with prior guidance that precision–recall views are especially informative for imbalanced binary classification [44, 45]. We therefore interpret recall together with precision, F1, and FP counts.

### 4.7. Implementation and Parameters

The reported CoDA-FTP configuration follows the method in Section 3. Source selection uses a fixed closest-quarter rule in which each leave-one-project-out fold selects the six closest projects from the 22 candidate source projects, with a minimum of three selected projects. The selected-source adaptation step uses a covariance regularization term of $10^{-3}$ for numerical stability. After adaptation, CoDA-FTP applies mild source-side imbalance handling with weak SMOTE using sampling ratio 0.075 and $k = 3$ neighbors.

The classifier is a regularized XGBoost model [42]. Unless otherwise stated, CoDA-FTP uses 250 trees, scale positive weight 3, maximum tree depth 3, minimum child weight 10, gamma 5, subsample 0.8, column subsampling 0.8, $\ell_2$ regularization 10, and fixed decision threshold 0.65. We use 0.65 as a conservative operating point, not as a per-target threshold learned from held-out target labels. Parameter choices can materially affect defect-prediction conclusions [52]; we therefore treat these choices as part of the reported protocol and analyze the sensitivity of the source-set size and decision threshold in RQ4. RQ4 also includes a source-only validation check for the source-set size.

### 4.8. Ablation and Sensitivity Analysis

For RQ3, we compare the full CoDA-FTP pipeline with variants that remove one or more cross-project design choices, including target-aware source selection, selected-source adaptation, regularized XGBoost, and both source selection and selected-source adaptation. These variants test whether conservative selected-source behavior comes from a single component or from the interaction between source selection, adaptation, and classifier regularization. We also include a Random-6 diagnostic that randomly selects six source projects for each target while keeping CORAL, XGBoost, imbalance handling, and the decision threshold unchanged. We repeat Random-6 20 times to distinguish target-aware source selection from merely reducing the amount of source data.

For RQ4, we evaluate sensitivity to the size of the selected source set and the decision threshold. The goal is



This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6963911

**Table 3**
Per-project performance comparison for traditional/neural baselines and CoDA-FTP under leave-one-project-out evaluation. Each method reports the full confusion matrix and derived precision, recall, and F1. Percentages are rounded to integers.

| | FlakeFlagger | | | | | | | Vocab.+FF | | | | | | | Flakify | | | | | | | DeepFlaky | | | | | | | CoDA-FTP | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Project | TP | FN | FP | TN | P | R | F1 | TP | FN | FP | TN | P | R | F1 | TP | FN | FP | TN | P | R | F1 | TP | FN | FP | TN | P | R | F1 | TP | FN | FP | TN | P | R | F1 |
| achilles | 0 | 4 | 5 | 1,307 | 0% | 0% | 0% | 0 | 4 | 0 | 1,312 | 0% | 0% | 0% | 0 | 4 | 0 | 1,312 | 0% | 0% | 0% | 0 | 4 | 11 | 1,301 | 0% | 0% | 0% | 0 | 4 | 0 | 1,312 | 0% | 0% | 0% |
| activiti | 2 | 30 | 59 | 1,950 | 3% | 6% | 4% | 0 | 32 | 12 | 1,997 | 0% | 0% | 0% | 14 | 18 | 292 | 1,717 | 5% | 44% | 8% | 26 | 6 | 474 | 1,535 | 5% | 81% | 10% | 0 | 32 | 16 | 1,993 | 0% | 0% | 0% |
| alluxio | 44 | 72 | 0 | 71 | 100% | 38% | 55% | 66 | 50 | 0 | 71 | 100% | 57% | 73% | 63 | 53 | 5 | 66 | 93% | 54% | 68% | 84 | 32 | 0 | 71 | 100% | 72% | 84% | 77 | 39 | 0 | 71 | 100% | 66% | 80% |
| ambari | 21 | 25 | 40 | 227 | 34% | 46% | 39% | 41 | 5 | 57 | 210 | 42% | 89% | 57% | 15 | 31 | 27 | 240 | 36% | 33% | 34% | 44 | 2 | 42 | 225 | 51% | 96% | 67% | 12 | 34 | 20 | 247 | 38% | 26% | 31% |
| assertj-core | 0 | 1 | 4 | 6,200 | 0% | 0% | 0% | 0 | 1 | 0 | 6,204 | 0% | 0% | 0% | 0 | 1 | 0 | 6,204 | 0% | 0% | 0% | 1 | 0 | 4,172 | 2,032 | 0% | 100% | 0% | 0 | 1 | 0 | 6,204 | 0% | 0% | 0% |
| commons-exec | 0 | 1 | 1 | 53 | 0% | 0% | 0% | 0 | 1 | 2 | 52 | 0% | 0% | 0% | 1 | 0 | 18 | 36 | 5% | 100% | 10% | 1 | 0 | 1 | 53 | 50% | 100% | 67% | 1 | 0 | 0 | 54 | 100% | 100% | 100% |
| elastic-job-lite | 0 | 3 | 0 | 555 | 0% | 0% | 0% | 0 | 3 | 0 | 555 | 0% | 0% | 0% | 1 | 2 | 21 | 534 | 5% | 33% | 8% | 3 | 0 | 10 | 545 | 23% | 100% | 38% | 3 | 0 | 11 | 544 | 21% | 100% | 35% |
| handlebars.java | 0 | 1 | 1 | 349 | 0% | 0% | 0% | 0 | 1 | 0 | 350 | 0% | 0% | 0% | 0 | 1 | 0 | 350 | 0% | 0% | 0% | 1 | 0 | 0 | 350 | 100% | 100% | 100% | 1 | 0 | 0 | 350 | 100% | 100% | 100% |
| hbase | 12 | 133 | 8 | 276 | 60% | 8% | 15% | 28 | 117 | 11 | 273 | 72% | 19% | 30% | 49 | 96 | 21 | 263 | 70% | 34% | 46% | 30 | 115 | 23 | 261 | 57% | 21% | 30% | 106 | 39 | 38 | 246 | 74% | 73% | 73% |
| hector | 0 | 33 | 0 | 98 | 0% | 0% | 0% | 0 | 33 | 0 | 98 | 0% | 0% | 0% | 9 | 24 | 20 | 78 | 31% | 27% | 29% | 0 | 33 | 0 | 98 | 0% | 0% | 0% | 31 | 2 | 9 | 89 | 78% | 94% | 85% |
| http-request | 0 | 18 | 0 | 145 | 0% | 0% | 0% | 0 | 18 | 0 | 145 | 0% | 0% | 0% | 0 | 18 | 19 | 126 | 0% | 0% | 0% | 1 | 17 | 0 | 145 | 100% | 6% | 11% | 5 | 13 | 0 | 145 | 100% | 28% | 43% |
| httpcore | 0 | 22 | 3 | 687 | 0% | 0% | 0% | 5 | 17 | 7 | 683 | 42% | 23% | 29% | 9 | 13 | 37 | 653 | 20% | 41% | 26% | 14 | 8 | 2 | 688 | 88% | 64% | 74% | 14 | 8 | 10 | 680 | 58% | 64% | 61% |
| incubator-dubbo | 0 | 19 | 13 | 2,101 | 0% | 0% | 0% | 0 | 19 | 0 | 2,114 | 0% | 0% | 0% | 6 | 13 | 54 | 2,060 | 10% | 32% | 15% | 17 | 2 | 5 | 2,109 | 77% | 89% | 83% | 16 | 3 | 1 | 2,113 | 94% | 84% | 89% |
| java-websocket | 0 | 23 | 0 | 122 | 0% | 0% | 0% | 0 | 23 | 0 | 122 | 0% | 0% | 0% | 20 | 3 | 1 | 121 | 95% | 87% | 91% | 1 | 22 | 1 | 121 | 50% | 4% | 8% | 0 | 23 | 0 | 122 | 0% | 0% | 0% |
| logback | 0 | 22 | 1 | 780 | 0% | 0% | 0% | 0 | 22 | 0 | 781 | 0% | 0% | 0% | 5 | 17 | 13 | 768 | 28% | 23% | 25% | 18 | 4 | 6 | 775 | 75% | 82% | 78% | 4 | 18 | 7 | 774 | 36% | 18% | 24% |
| ninja | 0 | 1 | 0 | 306 | 0% | 0% | 0% | 0 | 1 | 2 | 304 | 0% | 0% | 0% | 1 | 0 | 14 | 292 | 7% | 100% | 12% | 0 | 1 | 0 | 306 | 0% | 0% | 0% | 1 | 0 | 3 | 303 | 25% | 100% | 40% |
| okhttp | 0 | 100 | 0 | 708 | 0% | 0% | 0% | 0 | 100 | 0 | 708 | 0% | 0% | 0% | 3 | 97 | 18 | 690 | 14% | 3% | 5% | 7 | 93 | 18 | 690 | 28% | 7% | 11% | 1 | 99 | 0 | 708 | 100% | 1% | 2% |
| orbit | 0 | 7 | 0 | 79 | 0% | 0% | 0% | 0 | 7 | 0 | 79 | 0% | 0% | 0% | 1 | 6 | 2 | 77 | 33% | 14% | 20% | 0 | 7 | 0 | 79 | 0% | 0% | 0% | 0 | 7 | 1 | 78 | 0% | 0% | 0% |
| spring-boot | 1 | 81 | 131 | 1,804 | 1% | 1% | 1% | 0 | 82 | 55 | 1,880 | 0% | 0% | 0% | 17 | 65 | 74 | 1,861 | 19% | 21% | 20% | 30 | 52 | 10 | 1,925 | 75% | 37% | 49% | 27 | 55 | 4 | 1,931 | 87% | 33% | 48% |
| undertow | 2 | 5 | 2 | 172 | 50% | 29% | 36% | 1 | 6 | 11 | 163 | 8% | 14% | 11% | 2 | 5 | 19 | 155 | 10% | 29% | 14% | 5 | 2 | 5 | 169 | 50% | 71% | 59% | 2 | 5 | 9 | 165 | 18% | 29% | 22% |
| wildfly | 0 | 23 | 32 | 938 | 0% | 0% | 0% | 0 | 23 | 16 | 954 | 0% | 0% | 0% | 4 | 19 | 114 | 856 | 3% | 17% | 6% | 5 | 18 | 52 | 918 | 9% | 22% | 12% | 5 | 18 | 6 | 964 | 45% | 22% | 29% |
| wro4j | 3 | 13 | 153 | 965 | 2% | 19% | 3% | 1 | 15 | 59 | 1,059 | 2% | 6% | 3% | 0 | 16 | 9 | 1,109 | 0% | 0% | 0% | 15 | 1 | 42 | 1,076 | 26% | 94% | 41% | 11 | 5 | 2 | 1,116 | 85% | 69% | 76% |
| zxing | 0 | 2 | 5 | 338 | 0% | 0% | 0% | 0 | 2 | 11 | 332 | 0% | 0% | 0% | 0 | 2 | 9 | 334 | 0% | 0% | 0% | 1 | 1 | 1 | 342 | 50% | 50% | 50% | 0 | 2 | 1 | 342 | 0% | 0% | 0% |
| **Total** | 85 | 639 | 458 | 20,231 | 16% | 12% | 13% | 142 | 582 | 243 | 20,446 | 37% | 20% | 26% | 220 | 504 | 787 | 19,902 | 22% | 30% | 25% | 304 | 420 | 4,875 | 15,814 | 6% | 42% | 10% | **317** | **407** | **138** | **20,551** | **70%** | **44%** | **54%** |

not to tune the reported protocol, but to check whether the main conclusion remains stable under reasonable changes to deployment choices.

## 5. Results

### 5.1. RQ1: How Effective Is CoDA-FTP Against Traditional and Neural Flaky-Test Predictors?

Table 3 reports complete per-project confusion matrices and derived metrics for the traditional and neural baselines. Although the table includes all raw counts for reproducibility, the analysis below focuses on the deployment-critical trade-off among false positives, precision, recall, and F1. The final row gives aggregate results across all held-out projects.

**Overall effectiveness.** The baselines show different failure modes under leave-one-project-out evaluation. FlakeFlagger is cautious, producing 458 false positives, but it recovers only 85 of the 724 flaky tests. Vocabulary + FlakeFlagger improves the signal by recovering 142 flaky tests and raising F1 from 13% to 26%, yet it still misses most flaky tests in the target projects. Flakify moves in the opposite direction by recovering 220 flaky tests but reporting 1,007 tests as flaky in total, of which 787 are false positives. DeepFlaky recovers more flaky tests than Flakify, with 304 true positives, but it does so by issuing 5,179 flaky warnings, including 4,875 false positives. Its aggregate precision is therefore only 6%, and its aggregate F1 drops to 10%. This low aggregate precision does not mean that DeepFlaky fails to predict positives on most projects; rather, it predicts positives broadly, and several large false-positive bursts, especially on *assertj-core*, dominate the corpus-level count.

CoDA-FTP improves this trade-off by increasing the number of true positives without expanding the positive set as aggressively as the baselines. Compared with Vocabulary + FlakeFlagger, CoDA-FTP identifies 175 additional flaky tests while reducing false positives by 105, raising recall from 20% to 44% and F1 from 26% to 54%. Compared with Flakify, CoDA-FTP identifies 97 additional flaky tests and reduces false positives by 649, raising precision from 22% to 70%. Compared with DeepFlaky, CoDA-FTP identifies 13 additional flaky tests while reducing false positives by 4,737. Put differently, 317 of the 455 tests reported by CoDA-FTP are truly flaky, whereas only 304 of the 5,179 tests reported by DeepFlaky are true positives. This difference is central to cross-project deployment because a predictor that transfers too aggressively may increase recall, but it also turns prediction into a large manual triage task.

The FLAST-style static-neighbor reproduction is substantially weaker in the same 23-project leave-one-project-out setting. Across the reproduced $k \in \{3, 7\}$ and $\sigma \in \{0.5, 0.95\}$ settings, it detects at most 12 flaky tests, reaches at most 3% F1, and obtains 54–56% AUC. This does not contradict FLAST's original use case, which is a fast static similarity method that can be tuned when local flaky examples are available. Rather, it shows that direct nearest-neighbor similarity over tokenized test code is too sparse for our cross-project deployment setting, where the held-out target project may not share close flaky-code neighbors with the source projects.

**Project-level behavior.** The improvement is not uniform across all target projects. CoDA-FTP performs particularly well on projects such as *hbase*, *hector*, *httpcore*, *incubator-dubbo*, *spring-boot*, and *wro4j*, where it recovers more flaky tests than the feature-based baselines while keeping false positives small relative to the neural baselines. These are the cases in which the select-then-align design appears most useful because enough source signal is available, but direct transfer would still over-predict the flaky class. At the same time, CoDA-FTP remains limited on some targets, such as *activiti*, *java-websocket*, *okhttp*, and *orbit*. These failures are important because they show that conservative selected-source adaptation is not equivalent to uniformly improving every project; when the selected sources do not contain useful analogues for the target project's flaky behavior, CoDA-FTP can still miss flaky tests. We therefore treat AUC as a secondary ranking diagnostic and report pooled score-based AUC in Table 5.



This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6963911

**Table 4**
Project-level summary for RQ1. Percentages are rounded to integers. Macro metrics average per-project scores, treating undefined precision or F1 as zero.

| Method | Agg. F1 | Macro Prec. | Macro Rec. | Macro F1 | Median F1 | F1>0 |
|---|---|---|---|---|---|---|
| FlakeFlagger | 13% | 11% | 6% | 7% | 0% | 7/23 |
| Vocabulary + FlakeFlagger | 26% | 12% | 9% | 9% | 0% | 6/23 |
| FLAST-style ($k = 3, \sigma = 0.5$) | 3% | 9% | 1% | 2% | 0% | 5/23 |
| Flakify | 25% | 21% | 30% | 19% | 13% | 17/23 |
| DeepFlaky | 10% | 44% | 52% | 38% | 38% | 18/23 |
| **CoDA-FTP** | **54%** | **50%** | **44%** | **41%** | **35%** | **17/23** |

**Table 5**
Score-based diagnostics for probabilistic baselines. PR-AUC, pooled AUC, and recall under fixed false-positive budgets are computed from aggregated out-of-fold per-test scores.

| Method | PR-AUC | Pooled AUC | R@FP100 | R@FP150 | R@FP200 |
|---|---|---|---|---|---|
| FlakeFlagger | 9% | 71% | 4% | 6% | 6% |
| Vocab.+FF | 16% | 75% | 10% | 14% | 19% |
| Flakify | 15% | 63% | 7% | 12% | 15% |
| DeepFlaky | 8% | 62% | 5% | 5% | 5% |
| **CoDA-FTP** | **48%** | **92%** | **33%** | **45%** | **47%** |

**Aggregate gains are also visible at the project level.** Table 4 separates corpus-level performance from per-project averages. This distinction matters because aggregate metrics can be dominated by projects with many flaky tests or many false positives. CoDA-FTP remains the strongest method under aggregate F1 and slightly improves macro F1 over DeepFlaky, but the macro view also reveals a more nuanced picture in which DeepFlaky reaches 38% macro F1 and 52% macro recall, yet its aggregate F1 is only 10%. The discrepancy comes from severe false-positive concentration in large target projects, most visibly *assertj-core*, where DeepFlaky produces 4,172 false positives for one flaky test. The FLAST-style baseline has the opposite failure mode because it keeps false positives low, but its token-neighbor criterion rarely transfers enough flaky evidence across projects. CoDA-FTP has lower macro recall than DeepFlaky, but it keeps the warnings more actionable, with 70% aggregate precision and 138 aggregate false positives. Its median F1 is 35%, compared with 38% for DeepFlaky and 13% for Flakify; the median view therefore shows that DeepFlaky can perform well on some small projects, while the aggregate view exposes its warning burden on large projects. Thus, CoDA-FTP's main advantage is not merely detecting flaky tests on more projects; it improves the deployment-level precision–recall balance by avoiding large false-positive bursts.

**The score ranking tells the same deployment story.** Table 5 checks whether CoDA-FTP's advantage depends only on the fixed threshold. It does not. CoDA-FTP reaches 48% PR-AUC, compared with 8–16% for the baselines. DeepFlaky's pooled score AUC is 62%, whereas its reproduced project-level mean AUC is 81%; this gap indicates that its scores are not well calibrated across target projects, with high-confidence false positives concentrated in large projects. Under a fixed budget of 150 false positives across the 23 held-out projects, CoDA-FTP recovers 45% of flaky tests, while the strongest baseline recovers 14% and DeepFlaky recovers only 5%. This fixed-budget view is close to a deployment question in which developers can inspect only a limited number of warnings, and CoDA-FTP ranks substantially more true flaky tests near the top of the warning list.

**Table 6**
Aggregate comparison with prompt-only LLM predictors. Percentages are rounded to integers; LLM predictors produce binary prompt decisions, so AUC is omitted.

| Prompt | Model | TP | FN | FP | TN | P | R | F1 |
|---|---|---|---|---|---|---|---|---|
| Zero | Qwen2.5-1.5B | 724 | 0 | 20,679 | 10 | 3% | 100% | 7% |
| Zero | Qwen2.5-7B | 50 | 674 | 638 | 20,051 | 7% | 7% | 7% |
| Zero | Qwen3.6-35B | 65 | 659 | 277 | 20,412 | 19% | 9% | 12% |
| 4-shot | Qwen2.5-1.5B | 718 | 6 | 20,372 | 317 | 3% | 99% | 7% |
| 4-shot | Qwen2.5-7B | 65 | 659 | 381 | 20,308 | 15% | 9% | 11% |
| 4-shot | Qwen3.6-35B | 257 | 467 | 1,405 | 19,284 | 15% | 35% | 22% |
| Adapt. | **CoDA-FTP** | **317** | **407** | **138** | **20,551** | **70%** | **44%** | **54%** |

***Answer to RQ1:*** CoDA-FTP improves cross-project flaky-test prediction mainly by controlling false-positive amplification while preserving useful recall. Compared with DeepFlaky, CoDA-FTP reduces false positives from 4,875 to 138 and raises aggregate F1 from 10% to 54%, supporting the claim that cross-project prediction benefits from conservative selected-source adaptation rather than direct transfer of all available source data.

## 5.2. RQ2: How Does CoDA-FTP Compare with Prompt-Only LLM Predictors?

Before analyzing CoDA-FTP's internal mechanism, we first compare it with a lightweight modern LLM usage mode based on direct prompting of code LLMs without task-specific training. This placement is intentional. If direct prompting were already sufficient for cross-project flaky-test prediction, the need for a source-learning and adaptation pipeline would be weaker.

Table 6 compares CoDA-FTP with prompt-only LLM predictors under the same leave-one-project-out evaluation. This RQ is a bounded reference comparison rather than a claim that direct prompting is a complete LLM-based training pipeline. As described in Section 4, these LLM methods use zero-shot or source-only few-shot prompting, without fine-tuning on the full source-project corpus, retrieval over source tests, or calibration. We report aggregate results because the central question is whether this lightweight LLM usage mode is sufficient for cross-project flaky-test prediction.

**Zero-shot LLM prompting is poorly calibrated for this rare-event task.** The zero-shot results show that larger code models do not automatically yield reliable flaky-test predictors. Qwen2.5-Coder-1.5B labels almost every test as flaky, recovering all 724 flaky tests but also producing 20,679 false positives. Its 100% recall therefore reflects an almost all-positive prediction behavior rather than meaningful discrimination between flaky and non-flaky tests. Larger models are



This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6963911

more selective, but the selectivity mostly suppresses recall. Qwen3.6-35B improves precision to 19%, yet detects only 65 flaky tests and reaches 12% F1. These results suggest that general code understanding is insufficient without a mechanism that calibrates predictions to the cross-project flaky-test distribution.

**Source examples help the LLM recognize flakiness, but not transfer it safely.** Adding four source-project examples changes the behavior of the strongest LLM, increasing Qwen3.6-35B's recall from 9% to 35%. However, this comes with 1,405 false positives and only 15% precision. The examples appear to make the model more willing to label tests as flaky, but they do not tell the model which source-project patterns should be trusted for the current target project. In other words, few-shot prompting provides weak task exposure, but it does not perform source selection or distribution adaptation.

CoDA-FTP has a much smaller final classifier than these LLMs, but it is trained inside a task-specific cross-project learning pipeline. Its advantage comes from using labeled source projects to learn flaky-test signals, using target-project features to select relevant sources, and adapting the selected source distribution before training a conservative classifier. This explicit learning process yields 70% precision and 54% F1, with only 138 false positives. Thus, the result is not that a simple classifier is inherently stronger than an LLM, nor that LLM-based flaky-test prediction is ineffective in general. Rather, the result shows that direct prompt-only use of code LLMs does not replace task-specific source learning and cross-project adaptation in this setting.

***Answer to RQ2:*** Under the lightweight prompt-only LLM setting studied here, CoDA-FTP achieves stronger precision and F1 than zero-shot and source-only four-shot LLM predictors. The best prompt-only LLM result reaches 22% F1, whereas CoDA-FTP reaches 54% F1 with far fewer false positives. This comparison should be interpreted as evidence about direct prompting, not as an upper bound on fine-tuned, retrieval-augmented, or calibrated LLM approaches.

### 5.3. RQ3: Which Design Choices and Transfer Operators Explain CoDA-FTP's Cross-Project Behavior?

RQ3 examines two related mechanism questions. First, we remove individual design choices to test whether their failure modes match the role they are intended to play. Second, we replace the transfer operator while keeping the rest of the selected-source pipeline fixed. This second diagnostic is not intended to rank all possible alignment methods; it tests whether the select-then-align protocol remains the central mechanism when the adaptation operator changes.

**Component ablations.** Table 7 reports the component ablation and source-selection diagnostic results. Each ablation removes one cross-project design choice and checks whether the resulting failure mode matches its intended role. Random-6 further tests whether the benefit of source selection comes merely from using fewer source projects.

**Table 7**
Component ablation and source-selection diagnostic. Random-6 repeats the full pipeline 20 times with six randomly chosen source projects per target; Random-6 is reported as mean and standard deviation. Percentages are rounded to integers.

| Variant | TP | FN | FP | TN |
|---|---|---|---|---|
| **Full CoDA-FTP** | **317** | **407** | **138** | **20,551** |
| Random-6 mean | 129 | 595 | 665 | 20,024 |
| Random-6 std. | 36 | 36 | 921 | 921 |
| w/o selection | 157 | 567 | 185 | 20,504 |
| w/o adaptation | 346 | 378 | 2,141 | 18,548 |
| w/o reg. clf. | 230 | 494 | 145 | 20,544 |
| w/o selection & adapt. | 272 | 452 | 3,828 | 16,861 |

| Variant | Prec. | Rec. | F1 |
|---|---|---|---|
| **Full CoDA-FTP** | **70%** | **44%** | **54%** |
| Random-6 mean | 30% | 18% | 21% |
| Random-6 std. | 17% | 5% | 9% |
| w/o selection | 46% | 22% | 30% |
| w/o adaptation | 14% | 48% | 22% |
| w/o reg. clf. | 61% | 32% | 42% |
| w/o selection & adapt. | 7% | 38% | 12% |

**Table 8**
Alternative transfer diagnostics. All variants use the same fused features, XGBoost configuration, source-side imbalance handling, and fixed threshold. "All-source" uses all candidate source projects; "selected-source" uses CoDA-FTP's target-aware source selection.

| Transfer operator | TP | FN | FP | TN | Prec. | Rec. | F1 | AUC |
|---|---|---|---|---|---|---|---|---|
| All-source TCA | 234 | 490 | 163 | 20,526 | 59% | 32% | 42% | 81% |
| Selected-source weighting | 335 | 389 | 2,145 | 18,544 | 14% | 46% | 21% | 88% |
| Selected-source TCA | 301 | 423 | 115 | 20,574 | 72% | 42% | 53% | 91% |
| **Selected-source CORAL** | **317** | **407** | **138** | **20,551** | **70%** | **44%** | **54%** | **92%** |

**Source selection controls what can be transferred.** Removing target-aware source selection reduces TP from 317 to 157, recall from 44% to 22%, and F1 from 54% to 30%. FP also increases from 138 to 185, so the loss is not merely a more conservative boundary. Random-6 confirms that the gain is not just from using six projects, since across 20 repetitions it reaches only 20.6% F1 on average and has unstable FP counts (664.7 $\pm$ 920.8). Source selection is therefore a transferability step, not an efficiency step.

**Adaptation and classifier control how transfer is applied.** Removing adaptation produces the complementary failure mode in which TP rises from 317 to 346, but FP jumps from 138 to 2,141 and precision drops from 70% to 14%. Removing both selection and adaptation is worse, producing 3,828 false positives and only 7% precision. These results show why CoDA-FTP needs the select-then-align interaction, where source selection decides *what* to transfer and adaptation decides *how* the selected source distribution should be used. The regularized classifier also matters. Without it, TP drops to 230 and F1 to 42%, while FP remains similar, indicating that regularization helps retain transferable positives without expanding warnings.



This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6963911

**Table 9**
Mechanism measurements for source selection and selected-source adaptation. Distances are computed in the standardized fused feature space; covariance discrepancy is the normalized Frobenius distance between source and target covariance matrices.

| Measurement | Value |
|---|---|
| Mean nearest-source distance | 10.7 |
| Mean selected-source distance | 18.7 |
| Mean unselected-source distance | 33.4 |
| Median selected/unselected distance ratio | 0.56 |
| Mean farthest-source distance | 40.2 |
| Largest farthest/nearest distance ratio | 22.8 |
| Covariance discrepancy before adaptation | 46.3 |
| Covariance discrepancy after adaptation | 0.001 |
| Mean covariance-discrepancy reduction | > 99% |

**Alternative transfer operators after source selection.** Table 8 compares CORAL with alternative transfer diagnostics. All-source TCA reaches only 42% F1 and 81% AUC, showing that applying a generic adaptation method to all sources is not enough. Once the source set is fixed, TCA becomes much closer, with selected-source TCA reaching 53% F1 and 91% AUC, compared with 54% F1 and 92% AUC for selected-source CORAL. This supports a conservative interpretation in which CoDA-FTP's main mechanism is selected-source adaptation, not the claim that CORAL dominates every possible alignment method. The source-weighting variant recovers 335 flaky tests but produces 2,145 false positives, showing that target-likeness weighting alone can still over-expand the flaky region.

**The mechanism measurements match the intended roles of the two adaptation steps.** Table 9 quantifies the two distributional operations in CoDA-FTP. The mean selected-source distance is 18.7, compared with 33.4 for unselected sources, and the median selected/unselected distance ratio is 0.56. Thus, source selection substantially reduces coarse project-level distance. Adaptation then addresses residual feature-level mismatch, and the normalized covariance discrepancy drops from 46.3 before adaptation to 0.001 afterward. This large reduction is expected from CORAL, but it verifies that selected-source adaptation materially changes the training geometry before classifier learning.

**Failure cases show the boundary of distributional closeness.** CoDA-FTP remains limited on targets such as *activiti*, *java-websocket*, *okhttp*, and *orbit*. These failures are not explained simply by the absence of nearby sources or by a lack of flaky examples in the selected source set. For example, *java-websocket* has a selected/unselected distance ratio of 0.51 and 64 flaky tests in its selected sources, yet CoDA-FTP detects none of its 23 flaky tests. Similarly, *okhttp* has 215 flaky tests in the selected source projects, but only one target flaky test is detected. A closer score audit supports the same interpretation, since the maximum predicted probability among true flaky tests is only 0.037 for *java-websocket*, and the median predicted probability among true flaky tests is only 0.082 for *okhttp*. These cases show that centroid proximity is a useful project-level transferability signal, but it is not a guarantee of label-conditional flaky-pattern transfer.

***Answer to RQ3:*** CoDA-FTP's performance comes from the interaction between target-aware source selection, selected-source adaptation, and conservative classifier regularization. Random-6 performs far below target-aware top-6, showing that source selection is not merely reducing the amount of source data. All-source TCA also underperforms selected-source adaptation, while selected-source TCA is competitive with CORAL; together, these results support the broader mechanism claim that conservative selected-source adaptation is central to cross-project flaky-test prediction.

## 5.4. RQ4: How Stable Is CoDA-FTP Under Deployment Choices?

RQ4 examines whether CoDA-FTP's conclusions depend on one accidental parameter setting. We focus on two deployment choices that directly affect cross-project behavior, namely the number of selected source projects and the decision threshold. We first check the reported closest-quarter source-set size using only source projects, and then report held-out target sensitivity as a diagnostic validation rather than as the procedure used to choose $k$.

**Source-only validation selects the closest-quarter rule.** To check the source-set size without consulting held-out target labels, we performed 2,530 source-only validation evaluations. For each outer target project, we removed that target, held out each remaining source project in turn as a pseudo-target, and evaluated $k \in \{3, 4, 5, 6, 7\}$ with the same select-then-align pipeline and fixed threshold 0.65. The source-only procedure selects $k = 6$ for 17 of the 23 target projects; the remaining choices are $k = 3$ for three targets, $k = 4$ for one target, $k = 5$ for two targets, and $k = 7$ for none.

This result supports the fixed closest-quarter rule before using held-out target labels. A separate dynamic-$k$ protocol based on the source-only selected values yields 65% precision, 33% recall, 44% F1, and 88% AUC; its lower recall is concentrated in large flaky-rich targets where validation selects a narrower source set. We therefore keep the main configuration simple and uniform across target projects.

**Held-out sensitivity confirms that the source set should be selective, not maximal.** After source-only validation identifies $k = 6$ as the most frequent choice, Table 10 checks how this choice behaves on held-out targets. This table uses target labels and is therefore a diagnostic sensitivity analysis, not the selection protocol. Smaller source sets remain precise but miss many flaky tests, with $k = 3$ and $k = 4$ reaching 45% F1 while $k = 5$ falls to 34% F1. With $k = 6$, CoDA-FTP recovers 317 flaky tests and reaches 54% F1 at 70% precision. Adding more source projects is not automatically beneficial, since $k = 7$ reaches only 35% F1, and $k = 10$ raises FP to 191 while recovering only 207 flaky tests.



This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6963911

**Table 10**
Held-out target sensitivity to the number of selected source projects. This table is a diagnostic validation of the source-only choice, not the procedure used to select $k$. Percentages are rounded to integers.

| Setting | TP | FN | FP | TN | Prec. | Rec. | F1 | AUC |
|---|---|---|---|---|---|---|---|---|
| $k = 3$ | 253 | 471 | 137 | 20,552 | 65% | 35% | 45% | 89% |
| $k = 4$ | 240 | 484 | 99 | 20,590 | 71% | 33% | 45% | 89% |
| $k = 5$ | 169 | 555 | 91 | 20,598 | 65% | 23% | 34% | 90% |
| **k = 6** | **317** | **407** | **138** | **20,551** | **70%** | **44%** | **54%** | **92%** |
| $k = 7$ | 185 | 539 | 146 | 20,543 | 56% | 26% | 35% | 83% |
| $k = 8$ | 176 | 548 | 132 | 20,557 | 57% | 24% | 34% | 86% |
| $k = 9$ | 190 | 534 | 159 | 20,530 | 54% | 26% | 35% | 89% |
| $k = 10$ | 207 | 517 | 191 | 20,498 | 52% | 29% | 37% | 88% |

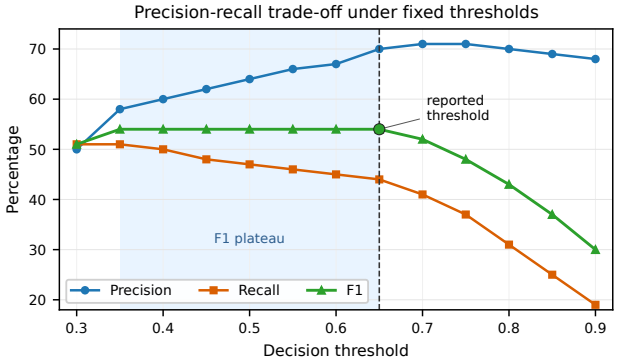



**Figure 3:** Precision–recall trade-off under fixed decision thresholds. The shaded region marks the threshold range in which CoDA-FTP maintains 54% F1; the dashed line marks the reported conservative operating point.

Thus, the closest-quarter rule is an interior operating point consistent with conservative source selection.

**The decision threshold changes the operating point, not the ranking conclusion.** Figure 3 visualizes the threshold scan for the reported $k = 6$ configuration. As the threshold increases, CoDA-FTP issues fewer warnings, precision increases, and recall decreases. At threshold 0.30, it recovers 372 flaky tests but produces 376 false positives; at threshold 0.90, it produces only 65 false positives but falls to 30% F1. The threshold therefore controls the deployment operating point rather than the ranking ability, and AUC remains 92% throughout the scan. We report 0.65 using a conservative operating point rule that prefers the threshold with fewer false positives among scanned thresholds whose unrounded F1 is within one percentage point of the best scanned F1.

The threshold scan is a post-hoc sensitivity analysis, not the procedure used to learn a threshold for each target project. From 0.35 to 0.65, CoDA-FTP maintains 54% F1 after rounding; over the same range, false positives decrease from 266 to 138 and precision increases from 58% to 70%. Beyond 0.65, additional conservatism no longer preserves the same balance, with F1 dropping to 52% at 0.70 and 48% at 0.75. Thus, the reported operating point lies at the high-precision end of the observed F1 plateau, but remains a fixed protocol choice rather than an automatically calibrated deployment threshold.

***Answer to RQ4:*** CoDA-FTP is stable in ranking quality but sensitive to overly broad source transfer. Source-only validation selects $k = 6$ for 17 of 23 target projects, supporting the fixed closest-quarter source-selection rule without using target labels for that check. The reported threshold of 0.65 is a fixed conservative operating point at the high-precision end of the near-optimal F1 region; the threshold scan explains this operating point but should not be read as a target-label threshold-selection protocol.

## 6. Threats to Validity

We organize threats using standard validity categories for empirical software engineering studies [53].

**External validity.** Our evaluation uses the public FlakeFlagger Java dataset [9]. The dataset provides project-level test instances, expert flaky-test features, and a common basis for comparison with prior flaky-test prediction work, but it still represents one dataset family consisting of Java projects, a particular collection process, and a specific set of open-source systems. The results may not directly generalize to other languages, industrial repositories, build systems, or projects whose flaky tests arise from different infrastructure and concurrency patterns. We mitigate this threat by using leave-one-project-out evaluation over 23 projects and scope the claim to cross-project flaky-test prediction within this dataset family.

The dataset is also highly imbalanced across projects. Aggregate confusion-matrix metrics reflect corpus-level deployment behavior, but they can be influenced by large projects and projects with many positives. We therefore report macro-level summaries, discuss project-level behavior, and identify targets where CoDA-FTP remains limited.

**Construct validity.** Flaky-test prediction depends on label quality. The dataset labels come from prior work and make the study reproducible [9], but they may still be affected by how flakiness was observed, how many reruns were used, and whether all nondeterministic behaviors were captured. Prior empirical work shows that flaky tests arise from diverse causes and can be difficult to expose consistently [1, 3, 4, 2]. Label errors would affect both CoDA-FTP and the baselines, and rare positives make this threat more visible on projects with few flaky tests.

Our metrics capture prediction behavior rather than developer cost, rerun savings, debugging time, or how teams would act on warnings [51, 44]. We emphasize precision and false positives because excessive warnings are a practical risk in rare-event prediction, but teams that assign higher cost to missed flaky tests may prefer a more recall-oriented operating point.

**Internal validity.** CoDA-FTP contains several design choices, including fused expert and semantic features, target-aware source selection, selected-source adaptation, weak source-side imbalance handling, and a regularized XGBoost



This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6963911

classifier. Implementation errors or preprocessing inconsistencies could affect the results. We reduce this risk by using the same leave-one-project-out split for all completed baselines, excluding project identifiers and test names from learning features, and reporting ablations that remove major components of the framework.

Parameter choices remain an internal threat. The reported protocol fixes the selected source-set size, decision threshold, and classifier hyperparameters. Prior defect-prediction research shows that parameter optimization can materially change model performance [52]. RQ4 mitigates this threat by studying source-set and threshold sensitivity and by adding a source-only validation check for the closest-quarter source-set size. The decision threshold and classifier hyperparameters remain fixed design choices rather than parameters automatically derived for each deployment; different operating points, calibration procedures, or hyperparameter choices may change the precision–recall trade-off.

CORAL, SMOTE, and XGBoost are not the only possible choices for adaptation, imbalance handling, and classification. We use them as a lightweight instantiation of conservative selected-source adaptation and evaluate alternative transfer diagnostics, but our results support the select-then-align design rather than the global optimality of this exact component combination.

**Conclusion validity.** The evaluation compares CoDA-FTP with traditional feature-based predictors, neural code-model baselines, transfer diagnostics, and prompt-only LLM reference methods. The LLM methods use zero-shot and source-only few-shot prompts rather than full source-project fine-tuning; this is relevant to direct LLM deployment, but it is not an upper bound on LLM-based flaky-test prediction.

Finally, our conclusions rely on aggregate performance, macro-level summaries, component ablations, Random-6 diagnostics, and alternative transfer diagnostics. We interpret the results as support for conservative selected-source adaptation in this dataset, not as a definitive ranking of all possible cross-project flaky-test prediction methods.

## 7. Conclusion

This paper studied cross-project flaky-test prediction as a deployment problem. In this setting, a predictor must transfer knowledge from existing projects to a target project whose tests may follow a different distribution. Our results show that this setting exposes a practical failure mode that can be hidden by mixed-project evaluation, where transferred predictors can over-expand the rare flaky class and produce many false positives.

We formulated conservative selected-source adaptation as a design principle and instantiated it in CoDA-FTP. CoDA-FTP first selects source projects that are close to the target project, then adapts only those selected sources to the target distribution, and finally trains a regularized classifier for target-project prediction. Under leave-one-project-out evaluation, CoDA-FTP achieves 70% precision, 44% recall, 54% F1, and 92% AUC, outperforming traditional feature-based predictors and neural code-model baselines in aggregate F1. A bounded reference comparison with zero-shot and source-only few-shot LLM prompting further shows that direct prompt-only use does not provide a competitive substitute for task-specific source learning and adaptation in this setting. Component ablations show that source selection, selected-source adaptation, and conservative classifier regularization address different parts of the cross-project transfer problem.

The sensitivity analysis further clarifies the method's deployment behavior. CoDA-FTP performs best with a moderate selected source set, and a source-only validation check selects the reported closest-quarter source-set size for most target projects. The reported threshold of 0.65 provides a fixed conservative operating point at the high-precision end of a near-optimal F1 region. These findings support the central conclusion that cross-project flaky-test prediction benefits from conservative selected-source adaptation that controls what is transferred and how it is adapted, rather than from direct transfer of all available historical data.

## CRediT authorship contribution statement

**Yu Zhang:** Writing - original draft, Visualization, Validation, Resources, Methodology, Formal analysis, Data curation, Conceptualization. **Yan Lei:** Writing - review and editing, Writing - original draft, Supervision, Conceptualization. **Huan Xie:** Writing - review and editing, Data curation.

## Data availability

The replication package of CoDA-FTP, including source code, scripts, precomputed features, and reproduction instructions, is available at https://github.com/yuxixixixii/CoDA-FTP/releases/tag/CoDA-FTP.

## Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## References

[1] Q. Luo, F. Hariri, L. Eloussi, D. Marinov, An empirical analysis of flaky tests, in: Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering, ACM, 2014, pp. 643–653. doi:10.1145/2635868.2635920.

[2] O. Parry, G. M. Kapfhammer, M. Hilton, P. McMinn, A survey of flaky tests, ACM Transactions on Software Engineering and Methodology 31 (1) (2022). doi:10.1145/3476105.

[3] W. Lam, R. Oei, A. Shi, D. Marinov, T. Xie, idflakies: A framework for detecting and partially classifying flaky tests, in: Proceedings of the 12th IEEE Conference on Software Testing, Validation and Verification, IEEE, 2019, pp. 312–322. doi:10.1109/ICST.2019.00038.

[4] W. Lam, K. Muslu, H. Sajnani, S. Thummalapenta, A study on the lifecycle of flaky tests, in: Proceedings of the 42nd International Conference on Software Engineering, ACM, 2020, pp. 1471–1482. doi:10.1145/3377811.3381749.



This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6963911

[5] A. Romano, Z. Song, S. Grandhi, W. Yang, W. Wang, An empirical analysis of ui-based flaky tests, in: Proceedings of the 43rd International Conference on Software Engineering, IEEE, 2021, pp. 1585–1597. doi:10.1109/ICSE43902.2021.00141.

[6] E. Kowalczyk, K. Nair, Z. Gao, L. Silberstein, T. Long, A. M. Memon, Modeling and ranking flaky tests at apple, in: Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering: Software Engineering in Practice, ACM, 2020, pp. 110–119. doi:10.1145/3377813.3381370.

[7] J. Bell, O. Legunsen, M. Hilton, L. Eloussi, T. Yung, D. Marinov, Deflaker: Automatically detecting flaky tests, in: Proceedings of the 40th International Conference on Software Engineering, ACM, 2018, pp. 433–444. doi:10.1145/3180155.3180164.

[8] O. Parry, G. M. Kapfhammer, M. Hilton, P. McMinn, Empirically evaluating flaky test detection techniques combining test case rerunning and machine learning models, Empirical Software Engineering 28 (2023). doi:10.1007/s10664-023-10307-w.

[9] A. Alshammari, C. Morris, M. Hilton, J. Bell, Flakeflagger: Predicting flakiness without rerunning tests, in: Proceedings of the 43rd International Conference on Software Engineering, IEEE, 2021, pp. 1572–1584. doi:10.1109/ICSE43902.2021.00140.

[10] S. Fatima, T. A. Ghaleb, L. Briand, Flakify: A black-box, language model-based predictor for flaky tests, IEEE Transactions on Software Engineering 49 (4) (2023) 1912–1927. doi:10.1109/TSE.2022.3201209.

[11] J. Cai, Y. Lei, Z. Li, C. Liu, H. Xie, Z. Wu, Deepflaky: Deep hybrid representation learning for flaky test prediction, Information and Software Technology 193 (2026) 108070. doi:10.1016/j.infsof.2026.108070.

[12] G. Haben, S. Habchi, J. Micco, M. Harman, M. Papadakis, M. Cordy, Y. L. Traon, The importance of accounting for execution failures when predicting test flakiness, in: Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, ACM, 2024, pp. 1979–1989. doi:10.1145/3691620.3695261.

[13] B. Turhan, T. Menzies, A. Bener, J. D. Stefano, On the relative value of cross-company and within-company data for defect prediction, Empirical Software Engineering 14 (5) (2009) 540–578. doi:10.1007/s10664-008-9103-7.

[14] Y. Ma, G. Luo, X. Zeng, A. Chen, Transfer learning for cross-company software defect prediction, Information and Software Technology 54 (3) (2012) 248–256. doi:10.1016/j.infsof.2011.09.007.

[15] J. Nam, S. J. Pan, S. Kim, Transfer defect learning, in: Proceedings of the 35th International Conference on Software Engineering, IEEE, 2013, pp. 382–391. doi:10.1109/ICSE.2013.6606584.

[16] J. Bai, J. Jia, L. F. Capretz, A three-stage transfer learning framework for multi-source cross-project software defect prediction, Information and Software Technology 150 (2022) 106985. doi:10.1016/j.infsof.2022.106985.

[17] H. Song, G. Wu, L. Ma, Y. Pan, Q. Huang, S. Jiang, Adversarial domain adaptation for cross-project defect prediction, Empirical Software Engineering 28 (2023). doi:10.1007/s10664-023-10371-2.

[18] S. Tabassum, L. L. Minku, D. Feng, Cross-project online just-in-time software defect prediction, IEEE Transactions on Software Engineering 49 (1) (2023) 268–287. doi:10.1109/TSE.2022.3150153.

[19] G. G. Cabral, L. L. Minku, A. L. I. Oliveira, D. A. Pessoa, S. Tabassum, An investigation of online and offline learning models for online just-in-time software defect prediction, Empirical Software Engineering 28 (2023). doi:10.1007/s10664-023-10335-6.

[20] S. Rahman, S. Dutta, A. Shi, Understanding and improving flaky test classification, Proceedings of the ACM on Programming Languages 9 (OOPSLA2) (2025). doi:10.1145/3763098.

[21] J. Nam, W. Fu, S. Kim, T. Menzies, L. Tan, Heterogeneous defect prediction, IEEE Transactions on Software Engineering 44 (9) (2018) 874–896. doi:10.1109/TSE.2017.2720603.

[22] S. Herbold, A. Trautsch, J. Grabowski, A comparative study to benchmark cross-project defect prediction approaches, IEEE Transactions on Software Engineering 44 (9) (2018) 811–833. doi:10.1109/TSE.2017.2724538.

[23] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, M. Zhou, Codebert: A pre-trained model for programming and natural languages, in: Findings of the Association for Computational Linguistics: EMNLP 2020, Association for Computational Linguistics, 2020, pp. 1536–1547. doi:10.18653/v1/2020.findings-emnlp.139.

[24] B. Sun, J. Feng, K. Saenko, Return of frustratingly easy domain adaptation, in: Proceedings of the 30th AAAI Conference on Artificial Intelligence, AAAI Press, 2016, pp. 2058–2065. doi:10.1609/aaai.v30i1.10306.

[25] S. Ben-David, J. Blitzer, K. Crammer, A. Kulesza, F. Pereira, J. W. Vaughan, A theory of learning from different domains, Machine Learning 79 (1–2) (2010) 151–175. doi:10.1007/s10994-009-5152-4.

[26] S. J. Pan, Q. Yang, A survey on transfer learning, IEEE Transactions on Knowledge and Data Engineering 22 (10) (2010) 1345–1359. doi:10.1109/TKDE.2009.191.

[27] Y. Ganin, E. Ustinova, H. Ajakan, P. Germain, H. Larochelle, F. Laviolette, M. Marchand, V. Lempitsky, Domain-adversarial training of neural networks, Journal of Machine Learning Research 17 (59) (2016) 1–35.
URL https://jmlr.org/papers/v17/15-239.html

[28] S. J. Pan, I. W. Tsang, J. T. Kwok, Q. Yang, Domain adaptation via transfer component analysis, IEEE Transactions on Neural Networks 22 (2) (2011) 199–210. doi:10.1109/TNN.2010.2091281.

[29] M. Long, Y. Cao, J. Wang, M. I. Jordan, Learning transferable features with deep adaptation networks, in: Proceedings of the 32nd International Conference on Machine Learning, Vol. 37 of Proceedings of Machine Learning Research, PMLR, 2015, pp. 97–105.
URL https://proceedings.mlr.press/v37/long15.html

[30] A. Gretton, K. M. Borgwardt, M. J. Rasch, B. Schoelkopf, A. Smola, A kernel two-sample test, Journal of Machine Learning Research 13 (2012) 723–773.
URL https://www.jmlr.org/papers/v13/gretton12a.html

[31] F. Peters, T. Menzies, A. Marcus, Better cross company defect prediction, in: Proceedings of the 10th Working Conference on Mining Software Repositories, IEEE, 2013, pp. 409–418. doi:10.1109/MSR.2013.6624057.

[32] G. Canfora, A. D. Lucia, M. D. Penta, R. Oliveto, A. Panichella, S. Panichella, Multi-objective cross-project defect prediction, in: Proceedings of the 6th IEEE International Conference on Software Testing, Verification and Validation, IEEE, 2013, pp. 252–261. doi:10.1109/ICST.2013.38.

[33] T. Zimmermann, N. Nagappan, H. Gall, E. Giger, B. Murphy, Cross-project defect prediction: A large scale experiment on data vs. domain vs. process, in: Proceedings of the 7th Joint Meeting of the European Software Engineering Conference and the ACM SIGSOFT Symposium on the Foundations of Software Engineering, ACM, 2009, pp. 91–100. doi:10.1145/1595696.1595713.

[34] X. Xia, D. Lo, S. J. Pan, N. Nagappan, X. Wang, HYDRA: Massively compositional model for cross-project defect prediction, IEEE Transactions on Software Engineering 42 (10) (2016) 977–998. doi:10.1109/TSE.2016.2543218.

[35] S. Hosseini, B. Turhan, D. Gunarathna, A systematic literature review and meta-analysis on cross project defect prediction, IEEE Transactions on Software Engineering 45 (2) (2019) 111–147. doi:10.1109/TSE.2017.2770124.

[36] S. Pal, A. Sillitti, Cross-project defect prediction: A literature review, IEEE Access 10 (2022) 118697–118717. doi:10.1109/ACCESS.2022.3221184.

[37] Z. He, F. Peters, T. Menzies, Y. Yang, Learning from open-source projects: An empirical study on defect prediction, in: Proceedings of the 2013 International Symposium on Empirical Software Engineering and Measurement, IEEE, 2013, pp. 45–54. doi:10.1109/ESEM.2013.20.

[38] X. Jing, F. Wu, X. Dong, F. Qi, B. Xu, Heterogeneous cross-company defect prediction by unified metric representation and CCA-based transfer learning, in: Proceedings of the 10th Joint Meeting on Foundations of Software Engineering, ACM, 2015, pp. 496–507. doi:




This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6963911

10.1145/2786805.2786813.

[39] J. Wang, Y. Lei, M. Li, G. Ren, H. Xie, S. Jin, J. Li, J. Hu, FLAKYRANK: Predicting flaky tests using augmented learning to rank, in: Proceedings of the 31st IEEE International Conference on Software Analysis, Evolution and Reengineering, IEEE, 2024, pp. 872–883. doi:10.1109/SANER60148.2024.00095.

[40] Z. Li, Z. Wu, Y. Lei, H. Xie, M. Li, J. Hu, Hiflaky: Hierarchy-aware flakiness classification, Journal of Systems and Software 234 (2026) 112741. doi:10.1016/j.jss.2025.112741.

[41] H. Xie, Y. Lei, M. Li, M. Yan, S. Zhang, Combining coverage and expert features with semantic representation for coincidental correctness detection, in: Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, ACM, 2024, pp. 1770–1782. doi:10.1145/3691620.3695542.

[42] T. Chen, C. Guestrin, Xgboost: A scalable tree boosting system, in: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, ACM, 2016, pp. 785–794. doi:10.1145/2939672.2939785.

[43] N. V. Chawla, K. W. Bowyer, L. O. Hall, W. P. Kegelmeyer, Smote: Synthetic minority over-sampling technique, Journal of Artificial Intelligence Research 16 (2002) 321–357. doi:10.1613/jair.953.

[44] J. Davis, M. Goadrich, The relationship between precision-recall and ROC curves, in: Proceedings of the 23rd International Conference on Machine Learning, ACM, 2006, pp. 233–240. doi:10.1145/1143844.1143874.

[45] T. Saito, M. Rehmsmeier, The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets, PLOS ONE 10 (3) (2015) e0118432. doi:10.1371/journal.pone.0118432.

[46] G. Pinto, B. Miranda, S. Dissanayake, M. d'Amorim, C. Treude, A. Bertolino, What is the vocabulary of flaky tests?, in: Proceedings of the 17th International Conference on Mining Software Repositories, ACM, 2020, pp. 492–502. doi:10.1145/3379597.3387482.

[47] R. Verdecchia, E. Cruciani, B. Miranda, A. Bertolino, Know you neighbor: Fast static prediction of test flakiness, IEEE Access 9 (2021) 76119–76134. doi:10.1109/ACCESS.2021.3082424.

[48] B. Hui, J. Yang, Z. Cui, J. Yang, D. Liu, L. Zhang, T. Liu, J. Zhang, B. Yu, K. Lu, K. Dang, Y. Fan, Y. Zhang, A. Yang, R. Men, F. Huang, B. Zheng, Y. Miao, S. Quan, Y. Feng, X. Ren, X. Ren, J. Zhou, J. Lin, Qwen2.5-coder technical report, arXiv preprint arXiv:2409.12186 (2024). doi:10.48550/arXiv.2409.12186.

[49] A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv, C. Zheng, D. Liu, F. Zhou, F. Huang, F. Hu, H. Ge, H. Wei, H. Lin, J. Tang, J. Yang, J. Tu, J. Zhang, J. Yang, J. Yang, J. Zhou, J. Zhou, J. Lin, K. Dang, K. Bao, K. Yang, et al., Qwen3 technical report, arXiv preprint arXiv:2505.09388 (2025). doi:10.48550/arXiv.2505.09388.

[50] L. Breiman, Random forests, Machine Learning 45 (1) (2001) 5–32. doi:10.1023/A:1010933404324.

[51] T. Fawcett, An introduction to ROC analysis, Pattern Recognition Letters 27 (8) (2006) 861–874. doi:10.1016/j.patrec.2005.10.010.

[52] C. Tantithamthavorn, S. McIntosh, A. E. Hassan, K. Matsumoto, Automated parameter optimization of classification techniques for defect prediction models, in: Proceedings of the 38th International Conference on Software Engineering, ACM, 2016, pp. 321–332. doi:10.1145/2884781.2884857.

[53] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, A. Wesslén, Experimentation in Software Engineering, Springer, 2012. doi:10.1007/978-3-642-29044-2.

Preprint not peer reviewed



This preprint research paper has not been peer reviewed. Electronic copy available at: https://ssrn.com/abstract=6963911